# Berachain

## Security Review

Cantina Managed review by:
**R0bert**, Lead Security Researcher
**0xWeiss**, Security Researcher

April 24, 2026

# Contents


**1 Introduction** **2**
- 1.1 About Cantina . . . 2
- 1.2 Disclaimer . . . 2
- 1.3 Risk assessment . . . 2
  - 1.3.1 Severity Classification . . . 2

**2 Security Review Summary** **3**
- 2.1 Scope . . . 3

**3 Findings** **5**
- 3.1 Low Risk . . . 5
  - 3.1.1 `BlockRewardController` redeems the full payout instead of the shortfall . . . 5
  - 3.1.2 V2 initializer bypasses reward rate limit . . . 5
  - 3.1.3 Initial state updates of `rewardRate` are not tracked properly . . . 6
  - 3.1.4 Stale reward allocators can still queue new reward allocations . . . 6
  - 3.1.5 Stale validator commission can be activated after the operator is updated . . . 8
- 3.2 Informational . . . 9
  - 3.2.1 Shared-factory CREATE2 rollouts with post-deployment initialization can be seized . . . 9
  - 3.2.2 Non-atomic upgrade can break live reward notifications . . . 9
  - 3.2.3 Hardcoded WBERA address can drift from config . . . 11
  - 3.2.4 Raw distributor allowances can block WBERA-backed claims . . . 11
  - 3.2.5 The `SAFE_GAS_LIMIT` constant has been left unused after the upgrade . . . 12
  - 3.2.6 PR 107 fresh deployment omits required WBERA-era runtime wiring . . . 12
  - 3.2.7 System-call replay can double-process rewards . . . 13

# 1 Introduction

## 1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

## 1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

## 1.3 Risk assessment

| Severity level | Impact: High | Impact: Medium | Impact: Low |
|---|---|---|---|
| **Likelihood: high** | Critical | High | Medium |
| **Likelihood: medium** | High | Medium | Low |
| **Likelihood: low** | Medium | Low | Low |

### 1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

# 2 Security Review Summary

From Apr 9th to Apr 12th the Cantina team conducted a review of Berachain Corporation on commit hash 720d9303. The team identified a total of **12** issues:

**Issues Found**

| Severity | Count | Fixed | Acknowledged |
|---|---|---|---|
| Critical Risk | 0 | 0 | 0 |
| High Risk | 0 | 0 | 0 |
| Medium Risk | 0 | 0 | 0 |
| Low Risk | 5 | 2 | 3 |
| Gas Optimizations | 0 | 0 | 0 |
| Informational | 7 | 3 | 4 |
| **Total** | **12** | **5** | **7** |

## 2.1 Scope

The security review had the following components in scope for Berachain Corporation on commit hash 720d9303:

```
src
├── base
│   ├── Create2Deployer.sol
│   ├── DeployHelper.sol
│   ├── EIP2612.sol
│   ├── EIP3009.sol
│   ├── FactoryOwnable.sol
│   ├── IStakingRewards.sol
│   ├── IStakingRewardsErrors.sol
│   ├── Salt.sol
│   └── StakingRewards.sol
└── pol
    ├── BeaconDeposit.sol
    ├── BeaconRootsHelper.sol
    ├── BGT.sol
    ├── BGTDeployer.sol
    ├── BGTFeeDeployer.sol
    ├── BGTIncentiveDistributorDeployer.sol
    ├── BGTIncentiveFeeCollector.sol
    ├── BGTIncentiveFeeDeployer.sol
    ├── BGTStaker.sol
    ├── DedicatedEmissionStreamManagerDeployer.sol
    ├── FeeCollector.sol
    ├── POLDeployer.sol
    ├── RewardAllocatorFactoryDeployer.sol
    ├── RewardVaultHelperDeployer.sol
    ├── WBERAStakerVault.sol
    ├── WBERAStakerVaultWithdrawalRequest.sol
    ├── WBERAStakerWithdrawReqDeployer.sol
    ├── lst
    │   ├── InfraredBeraAdapter.sol
    │   ├── LSTStakerVault.sol
    │   ├── LSTStakerVaultFactory.sol
    │   ├── LSTStakerVaultFactoryDeployer.sol
    │   └── LSTStakerVaultWithdrawalRequest.sol
    └── rewards
        ├── BeraChef.sol
        ├── BGTIncentiveDistributor.sol
        ├── BlockRewardController.sol
```

```
        ├── DedicatedEmissionStreamManager.sol
        ├── Distributor.sol
        ├── RewardAllocatorFactory.sol
        ├── RewardVault.sol
        ├── RewardVaultFactory.sol
        └── RewardVaultHelper.sol
```

# 3 Findings

## 3.1 Low Risk

### 3.1.1 `BlockRewardController` redeems the full payout instead of the shortfall

**Severity:** Low Risk

**Context:** BlockRewardController.sol#L192-L200

**Description:** `_handleMinting` checks `address(this).balance < amount`, but when that condition is true it mints and redeems the full `amount` of `BGT`. It does not subtract the native balance the controller already holds and redeem only the missing portion.

This matters because `BGT.mint` is limited by `BGT`'s own native reserve invariant, not by the controller's balance. The invariant only checks whether `address(bgt).balance >= totalSupply()` after minting. Native BERA already sitting on `BlockRewardController` helps pay the final `WBERA` transfer, but it does not increase how much `BGT` is allowed to mint first. Consequently, `processRewards` can revert even when the controller balance plus the remaining `BGT` reserve are enough to cover the payout in aggregate.

For example, assume the controller already holds `0.5` BERA and needs to deliver `1` WBERA. Only `0.5` more native token is actually missing. If `BGT` has reserve headroom for exactly that `0.5` shortfall, the payout should succeed. The current code still tries to mint and redeem `1`. `BGT.mint(1)` can therefore revert before the controller ever uses the `0.5` BERA it already had.

**Recommendation:** Redeem only the shortfall. Compute the controller's native balance first, then mint and redeem only `amount - balance` when that value is nonzero. For example:

```
uint256 balance = address(this).balance;
if (balance < amount) {
    uint256 shortfall = amount - balance;
    bgt.mint(address(this), shortfall);
    bgt.redeem(address(this), shortfall);
}
```

**Berachain:** Acknowledged. The contract was upgraded to be independent from the hard fork. Once the hard fork is completed, it is guaranteed by design that the `BlockRewardController` will always have sufficient native tokens to back the emissions. In this context, the conditional check could effectively act as a flag. However, it must account for potential attack scenarios where an attacker attempts to send native tokens to the contract before the hard fork occurs, reverting the behavior back to `BGT` minting.

**Cantina Managed:** Acknowledged.

### 3.1.2 V2 initializer bypasses reward rate limit

**Severity:** Low Risk

**Context:** BlockRewardController.sol#L93-L107

**Description:** `BlockRewardController.initialize(address payable _wbera, uint256 _rewardRate)` writes `rewardRate` directly and does not enforce `MAX_REWARD_RATE`. This differs from `setRewardRate`, which rejects out-of-range values with `InvalidRewardRate`.

Therefore the documented reward cap is not absolute. Governance can set an oversized initial emission rate during the upgrade by calling the V2 initializer through `upgradeToAndCall`, even though the normal admin setter forbids the same value. That breaks the assumptions used elsewhere in the codebase and tests that treat `MAX_REWARD_RATE` as the upper bound for live emissions.

**Recommendation:** Apply the same bound check in the V2 initializer before storing `_rewardRate` and revert with `InvalidRewardRate` when `_rewardRate > MAX_REWARD_RATE`.

**Berachain:** Fixed in commit 479d0e7.

**Cantina Managed:** Fix verified.

### 3.1.3 Initial state updates of `rewardRate` are not tracked properly

**Severity:** Low Risk

**Context:** BlockRewardController.sol#L102

**Description:** The `initialize` function is missing to track the initial state update for the `rewardRate` variable:

```
function initialize(address payable _wbera, uint256 _rewardRate) external
↪  reinitializer(VERSION) onlyOwner {
    if (_wbera == address(0)) {
        ZeroAddress.selector.revertWith();
    }

    wbera = IWBERA(_wbera);
    rewardRate = _rewardRate; // <<<

    _minBoostedRewardRate = 0;
    _boostMultiplier = 0;
    _rewardConvexity = 0;
}
```

The event `RewardRateChanged` should be emitted every time the reward rate is updated but it misses to be called in this instance, which could affect indexers.

**Recommendation:** Emit the `RewardRateChanged` event:

```
  function initialize(address payable _wbera, uint256 _rewardRate) external
  ↪  reinitializer(VERSION) onlyOwner {
        if (_wbera == address(0)) {
            ZeroAddress.selector.revertWith();
        }

        wbera = IWBERA(_wbera);
        rewardRate = _rewardRate;

        _minBoostedRewardRate = 0;
        _boostMultiplier = 0;
        _rewardConvexity = 0;

+     emit RewardRateChanged(rewardRate, _rewardRate);
     }
```

**Berachain:** Fixed in commit 388fa20.

**Cantina Managed:** Fix verified.

### 3.1.4 Stale reward allocators can still queue new reward allocations

**Severity:** Low Risk

**Context:** BeraChef.sol#L373

**Description:** Updating the operator for a public key by calling `requestOperatorChange` and `acceptOperatorChange` in the beacon deposit contract leaves a window for a stale reward allocator to call `queueNewRewardAllocation`.

In the `setValRewardAllocator` function, an operator from a certain validator public key, can add its reward allocator address:

```
function setValRewardAllocator(
    bytes calldata valPubkey,
    address rewardAllocator
)
    external
    onlyOperator(valPubkey) // <<<
{
```

```
    if (rewardAllocator == address(0)) {
        ZeroAddress.selector.revertWith();
    }
    valRewardAllocator[valPubkey] = rewardAllocator; // <<<
    emit ValRewardAllocatorSet(valPubkey, rewardAllocator);
}
```

If such operator wants to request an operator change by calling `requestOperatorChange` in the `BeaconDeposit.sol` contract and the change is accepted, the `rewardAllocator` will still be linked to the old operator who will still be allowed to queue new reward allocations:

```
function queueNewRewardAllocation(
    bytes calldata valPubkey,
    uint64 startBlock,
    Weight[] calldata weights
)
    external
{
    // get the reward allocator address.
    address rewardAllocator = valRewardAllocator[valPubkey]; // <<<
    if (rewardAllocator == address(0)) {
        // for backward compatibility, if the reward allocator is not set, use the
        ↪  operator address.
        rewardAllocator = beaconDepositContract.getOperator(valPubkey);
    }
    // only allow the reward allocator to queue a new reward allocation.
    if (msg.sender != rewardAllocator) {
        NotRewardAllocator.selector.revertWith();
    }
```

**Recommendation:** Do add the following changes to make sure an old allocator can't queue new reward allocations if the operator has been changed:

- Add a mapping to store the operator to a public key:

```
+ mapping(bytes valPubkey => address Operator) internal valOperator;
```

- Check that the current operator is the same one to the one that stored the rewards allocator:

```
  {
        // get the reward allocator address.
        address rewardAllocator = valRewardAllocator[valPubkey];
+        address currentOperator = beaconDepositContract.getOperator(valPubkey);
        if (rewardAllocator == address(0)) {
            // for backward compatibility, if the reward allocator is not set, use the
            ↪  operator address.
-            rewardAllocator = beaconDepositContract.getOperator(valPubkey);
+            rewardAllocator = currentOperator;
+        } else if (valOperator[valPubkey] != currentOperator) {
+            if (msg.sender != currentOperator) {
+                NotOperator.selector.revertWith();
+            }
+            rewardAllocator = currentOperator;
        }
```

- Store the operator for such public key:

```
  function setValRewardAllocator(
        bytes calldata valPubkey,
        address rewardAllocator
        )external onlyOperator(valPubkey)    {

          if (rewardAllocator == address(0)) {
            ZeroAddress.selector.revertWith();
        }
        valRewardAllocator[valPubkey] = rewardAllocator;
```

```
+         valOperator[valPubkey] = msg.sender;
          emit ValRewardAllocatorSet(valPubkey, rewardAllocator);
      }
```

**Berachain:** Acknowledged. The finding describes a defensive programming gap that does not constitute an exploitable vulnerability. The sole trigger for this scenario is the current operator calling `requestOperatorChange()`. There is no forced revocation, no governance override, and no third-party path to initiate an operator transfer. The potential attacker would have to voluntarily relinquish their own role, which cannot be considered an external threat, but rather one side of a cooperative handover acting in bad faith. The invariant gap is acknowledged and will be addressed internally.

**Cantina Managed:** Acknowledged.

### 3.1.5 Stale validator commission can be activated after the operator is updated

**Severity:** Low Risk

**Context:** BeraChef.sol#L336

**Description:** Stale Validator commission can be activated after the operator is updated. Calling `queueValCommission` allows the current operator for a certain validator to queue the commission rate which later needs to be accepted after the `commissionChangeDelay` goes by.

```
function queueValCommission(bytes calldata valPubkey, uint96 commissionRate) external
↪   onlyOperator(valPubkey) {
    if (commissionRate > MAX_COMMISSION_RATE) {
        InvalidCommissionValue.selector.revertWith();
    }
    QueuedCommissionRateChange storage qcr = valQueuedCommission[valPubkey];

    if (qcr.blockNumberLast > 0) {
        CommissionChangeAlreadyQueued.selector.revertWith(); //ok
    }
    (qcr.blockNumberLast, qcr.commissionRate) = (uint32(block.number), commissionRate);
    ↪  // <<<
    emit QueuedValCommission(valPubkey, commissionRate);
}
```

On activation, there is no check for an operator to be the `msg.sender`, therefore if the operator is changed while the `CommissionRate` gets active, then anyone could activate the stale commission rate for the new operator.

```
function activateQueuedValCommission(bytes calldata valPubkey) external { // <<<
    QueuedCommissionRateChange storage qcr = valQueuedCommission[valPubkey];
    (uint32 blockNumberLast, uint96 commissionRate) = (qcr.blockNumberLast,
    ↪  qcr.commissionRate);
    uint32 activationBlock = uint32(blockNumberLast + commissionChangeDelay); // <<<
    if (blockNumberLast == 0 || block.number < activationBlock) {
        CommissionNotQueuedOrDelayNotPassed.selector.revertWith();
    }
    uint96 oldCommission = _getOperatorCommission(valPubkey);
    valCommission[valPubkey] = CommissionRate({ activationBlock: activationBlock,
    ↪  commissionRate: commissionRate });
    emit ValCommissionSet(valPubkey, oldCommission, commissionRate);
    // delete the queued commission
    delete valQueuedCommission[valPubkey];
}
```

**Recommendation:** Allow the operator to update the commission rate if it has not been activated yet:

```
  }
          QueuedCommissionRateChange storage qcr = valQueuedCommission[valPubkey];
-         if (qcr.blockNumberLast > 0) {
-             CommissionChangeAlreadyQueued.selector.revertWith();
-         }
```

```
        (qcr.blockNumberLast, qcr.commissionRate) = (uint32(block.number),
        ↪  commissionRate);
        emit QueuedValCommission(valPubkey, commissionRate);
```

and check whether the msg.sender that calls `activateQueuedValCommission` is the operator too:

```
- function activateQueuedValCommission(bytes calldata valPubkey) external {
+   function activateQueuedValCommission(bytes calldata valPubkey) external
↪  onlyOperator(valPubkey) {
      QueuedCommissionRateChange storage qcr = valQueuedCommission[valPubkey];
```

**Berachain:** Acknowledged. The finding describes a defensive programming gap that does not constitute an exploitable vulnerability. The sole trigger for this scenario is the current operator calling `requestOperatorChange()`. There is no forced revocation, no governance override, and no third-party path to initiate an operator transfer. The potential attacker would have to voluntarily relinquish their own role, which cannot be considered an external threat, but rather one side of a cooperative handover acting in bad faith. The invariant gap is acknowledged and will be addressed internally.

**Cantina Managed:** Acknowledged.

## 3.2 Informational

### 3.2.1 Shared-factory CREATE2 rollouts with post-deployment initialization can be seized

**Severity:** Informational

**Context:** POLDeployer.sol#L46-L99

**Description:** `POLDeployer` uses the public `0x4e59...` deterministic deployment factory and deploys canonical implementations and proxies before it applies `governance` in later `initialize(...)` calls. This is unsafe because the proxy address is derived from the shared factory, the salt, and fixed proxy init code. The privileged values are not part of that address calculation. Changing `governance` therefore does not move the canonical proxy addresses.

An observer who sees the salts in the broadcast transaction can send the same deployments through the same shared factory first, then initialize the contracts with attacker-controlled owner or governance values. The official deployment then reverts because those `CREATE2` slots are already occupied, while the expected canonical addresses already point at attacker-controlled contracts. `_checkDeploymentAddress` only proves the address matches the prediction. It does not prove the owner, admin roles, or initializer parameters are correct.

This issue does not apply to every `CREATE2` deployment. If constructor arguments are part of the deployed init code, changing them changes the address. The risk here is specific to shared-factory deployments where privileged parameters are deferred to a later initializer. `BGTDeployer`, `BGTFeeDeployer`, `RewardVaultHelperDeployer`, `DedicatedEmissionStreamManagerDeployer`, `RewardAllocatorFactoryDeployer`, `HoneyDeployer`, and `GovDeployer` use the same pattern.

**Recommendation:** Do not broadcast these deployments through the public mempool. Use private orderflow for the full rollout, or deploy proxies with initialization calldata embedded in the proxy constructor so the privileged values are part of the `CREATE2` address derivation. Address checks alone are not enough. The rollout should also assert the actual owner, admin, and role state immediately after deployment.

**Berachain:** Acknowledged. During fresh deployments, a private mempool is used to prevent front-running. Additionally, a dedicated deployer contract is used to deploy implementations and proxies, and to perform initialization within a single atomic transaction.

**Cantina Managed:** Acknowledged.

### 3.2.2 Non-atomic upgrade can break live reward notifications

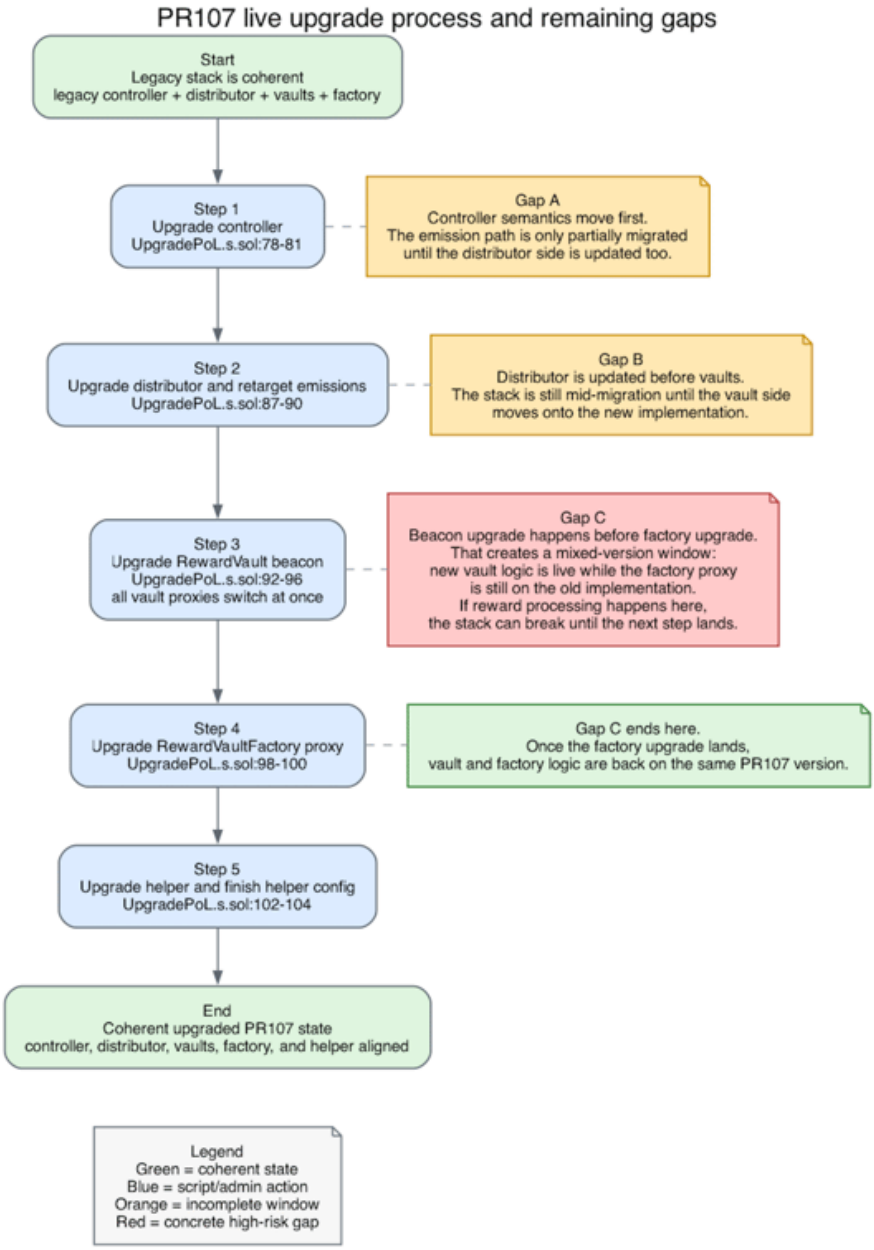
**Severity:** Informational

**Context:** UpgradePoL.s.sol#L78-L105

**Description:** `UpgradePoL.upgrade` applies the `BlockRewardController`, `Distributor`, `RewardVault` beacon and `RewardVaultFactory` changes in separate transactions. That ordering leaves live intermediate states where `distributeFor` can run against contracts that no longer agree on the reward token or the factory API they expect.

This happens in three concrete windows. After the controller upgrade, the controller starts sending `WBERA` while the old distributor still extends `BGT` allowance to vaults. After the distributor switch, old vaults still validate rewards against `BGT` allowance and revert when notified with `WBERA` funding. After the beacon upgrade, the new vault implementation calls `incentiveTokensCollector()`, but the old factory does not expose that selector yet. Therefore, on mainnet after September 3, 2025 16:00:00 UTC, a system-call to `Distributor.distributeFor(bytes)` during rollout can revert before that reward-notification attempt completes, or succeed while recording rewards against the wrong asset.

Pausing vaults does not solve this. `notifyRewardAmount` is still callable by the distributor even when user actions are paused, so reward notifications remain exposed until `distributeFor` itself is blocked or emissions are effectively zero.




**Recommendation:** Make the cutover atomic. If that is not possible, explicitly stop reward notifications for the whole rollout window by blocking `distributeFor` or driving effective emissions to zero before any upgrade transaction is sent. Only resume notifications after the factory, vault beacon, distributor token configuration, and controller are all in a mutually compatible state.

**Berachain:** Fixed in commit 69d557b.

**Cantina Managed:** Fix verified.

### 3.2.3 Hardcoded WBERA address can drift from config

**Severity:** Informational

**Context:** RewardVault.sol#L565-L571

**Description:** `RewardVault` hardcodes `WBERA_ADDRESS` and force-migrates `_rewardToken` to that constant. `RewardVaultHelper` uses the same hardcoded address for output-token validation, unwrapping, and sWBERA deposits. At the same time, `BlockRewardController.initialize(address payable _wbera, uint256 _rewardRate)` and `Distributor.setEmissionToken(address _emissionToken)` still accept arbitrary token addresses.

This leaves the system with two sources of truth for the emission token. If governance or an upgrade script ever points the controller or distributor at a different address, vault accounting and helper conversions will still assume the hardcoded `0x6969...6969` token. The current scripts and tests all use the canonical address, so this may stay hidden for now. It is still one configuration mistake away from a split state that would be difficult to diagnose after deployment.

**Recommendation:** Keep a single source of truth for WBERA. Consider rejecting any controller or distributor configuration that does not match the canonical address used by the vaults. Another option is removing the hardcoded constant from `RewardVault` and `RewardVaultHelper` and read the emission token from shared protocol configuration.

**Berachain:** Fixed in commit be145ad.

**Cantina Managed:** Fix verified.

### 3.2.4 Raw distributor allowances can block WBERA-backed claims

**Severity:** Informational

**Context:** RewardVault.sol#L599-L634

**Description:** `RewardVault` treats the distributor's raw `BGT` and `WBERA` allowances as if they were fully spendable. `_safeTransferRewardToken` enters the legacy `BGT` redemption step from `bgtAllowance` alone and `_checkRewardSolvency` accepts rewards from the sum of raw `BGT` and `WBERA` allowances. Neither check verifies that the distributor can still transfer those tokens.

This is weak after the migration. The legacy `BGT` leg depends on more than allowance. The distributor must still hold enough `BGT` and `BGT.transferFrom` still requires the distributor to remain an approved sender in `BGT`. If governance removes the distributor from the `BGT` whitelist while even a small residual `BGT` allowance remains, the vault still tries to pull `BGT` first and reverts with `NotApprovedSender` before it reaches the available `WBERA` amount. The same problem appears whenever approvals and balances diverge even though that is an invariant that is not supposed to break given the current code.

Once the distributor is removed from the `BGT` whitelist, the leftover approval can become hard to unwind. `BGT.approve` is also limited to approved senders, so a dewhitelisted distributor may no longer be able to reset its own allowance to zero. In practice, that stale approval can remain in place until governance temporarily re-whitelists the distributor or fixes the state another way.

**Recommendation:** Do not dewhitelist the live distributor until residual `BGT` approvals have been cleared. If that cleanup cannot be guaranteed, add an admin-controlled recovery step so stale approvals can be removed before the whitelist bit is dropped.

**Berachain:** Acknowledged. At the moment, governance based on `BGT` as a governance token is not yet live and the `BGT` token is expected to be deprecated. Shall we introduce a new governance system, the ownership of the deprecated `BGT` contract will not be transferred to it. As an additional security measure, a set of guardians will be in place to reject any proposals related to the `BGT` contract.

**Cantina Managed:** Acknowledged.

### 3.2.5 The `SAFE_GAS_LIMIT` constant has been left unused after the upgrade

**Severity:** Informational

**Context:** RewardVault.sol#L65

**Description:** The older logic in the `_processIncentives` function has been modified to not fetch `SAFE_GAS_LIMIT` anymore:

```
  if (amount > 0) {
-   // Transfer the remaining amount of the incentive to the bgtIncentiveDistributor
↪   contract for
-   // distribution among BGT boosters.
-   // give the bgtIncentiveDistributor the allowance to transfer the incentive token.
-   bytes memory data = abi.encodeCall(IERC20.approve, (bgtIncentiveDistributor,
↪   amount));
-   (bool success,) = token.call{ gas: SAFE_GAS_LIMIT }(data); // <<<
-   if (success) {
-     // reuse the already defined data variable to avoid stack too deep error.
```

Instead `TRANSFER_GAS_LIMIT` is called inside the `Utils` library.

**Recommendation:** Remove the `SAFE_GAS_LIMIT` constant:

```
- uint256 private constant SAFE_GAS_LIMIT = 500_000;
```

**Berachain:** Fixed in commit 8955cd8.

**Cantina Managed:** Fix verified.

### 3.2.6 PR 107 fresh deployment omits required WBERA-era runtime wiring

**Severity:** Informational

**Context:** *(No context files were provided by the reviewer)*

**Description:** PR 107 changed fresh deployment from a mostly complete bootstrap into a partial bootstrap. `3_DeployPoL.s.sol` still stops after the legacy `ConfigPOL` calls, but the upgraded reward stack now needs extra WBERA-era wiring before a new system can safely process rewards.

The missing work is split across several bootstrap steps. `POLDeployer` never runs the `BlockRewardController` V2 initializer, so `wbera` stays unset. The same flow leaves `Distributor.emissionToken` on legacy `BGT`. Fresh deployment also never sets `RewardVaultFactory.incentiveTokensCollector` and `10_DeployRewardVaultHelper.s.sol` never sets `RewardVaultHelper.sWBERA`. Even after those pointers are repaired, PR 107 reward emission still depends on `BGT` holding enough native BERA reserve and `2_DeployBGT.s.sol` does not fund that reserve on mainnet or devnet.

These gaps compose into one bootstrap failure rather than five isolated paper cuts. A clean deployment can succeed, ownership can be handed over and the system can still remain unable to emit WBERA rewards, unable to route protocol incentive tokens and unable to serve the new `sWBERA` claim mode. Local regression only becomes healthy once those manual follow-up calls and reserve funding steps are applied.

There are also two older bootstrap gaps that were already present before the PR 107. First, helper registration is still a separate step: governance must call `RewardVaultFactory.setRewardVaultHelper(...)` after deploying `RewardVaultHelper`, or helper-assisted claims remain disabled. Second, `BeraChef` still has older post-deploy defaults that the fresh-deploy scripts do not set, including `maxWeightPerVault`, `commissionChangeDelay`, `rewardAllocationInactivityBlockSpan` and baseline-factory wiring.

**Recommendation:** Make the fresh deployment flow complete before it is considered live. During bootstrap, run the `BlockRewardController` V2 initializer with canonical `WBERA`, switch `Distributor.emissionToken` to canonical `WBERA`, set `RewardVaultFactory.incentiveTokensCollector` to the intended live collector, set `RewardVaultHelper.sWBERA` to the live `WBERAStakerVault` and assert that the `BGT` native reserve is funded before nonzero emissions can be enabled. If these steps must remain split across scripts, add post-deploy assertions and fail the rollout until every dependency above is configured.

**Berachain:** Acknowledged. We will review all deployment scripts to ensure correctness for fresh deployments.

**Cantina Managed:** Acknowledged.

### 3.2.7 System-call replay can double-process rewards

**Severity:** Informational

**Context:** Distributor.sol#L164-L166

**Description:** `Distributor.distributeFor(bytes)` skips the replay check used by the proof-based `distributeFor(...)` overload. The proof-based function first marks `nextTimestamp` as processed through the beacon-root history buffer before it distributes rewards. The system-call overload does not do that. It passes `block.timestamp` straight into `_distributeFor`, and `BlockRewardController.processRewards` does not enforce its own "once per block timestamp" invariant either.

Consequently, if the execution-layer integration calls `distributeFor(bytes)` twice in the same block, both calls succeed and both process rewards. The second call is not rejected by any replay marker or per-block guard. This duplicates the validator base reward and the distributor-funded vault allocation for that block.

This is not reachable by an arbitrary user because of `onlySystemCall`. It is an integration risk because correctness is delegated to the privileged system caller without an onchain replay check. A client bug or duplicated hook can overpay emissions for a block instead of failing closed.

**Recommendation:** Add an explicit replay guard for the system entrypoint. A suggested fix is to store the last processed timestamp or block number for `distributeFor(bytes)` and revert if the same block is processed twice. A second guard inside `BlockRewardController.processRewards` would make the invariant hold even if another distributor bug or future integration change bypasses the outer check.

**Berachain:** Acknowledged. In the Berachain EVM fork, a check exists that validates that the PoL transaction is present in the block, that it is the first transaction, and that no other PoL transactions are included, see mod.rs#71 at commit cfd95915.

**Cantina Managed:** Acknowledged.